Data 37711 Homework 2 — Problem 3

Random Kitchen Sinks vs. a one-hidden-layer MLP

Yiwen Chen

The code is in `problem3.py`. It uses the provided `dataloader.py` and `plotting.py`; `models.py` was used as a template, and both models are implemented in the script. The data are 600 training points, 200 validation points, and 200 unlabelled test points, with $(x_1, x_2) \in [-3, 3]^2$. Predicting the training mean gives a validation MSE of 2.397, which is the baseline every model should beat by a wide margin.

(a) Data visualization

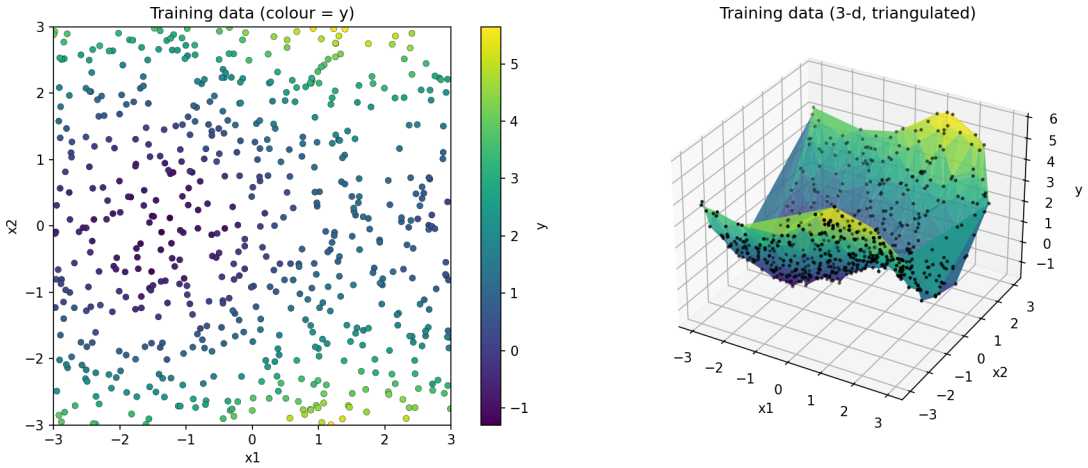


Figure 1: Training data. Left: points coloured by y. Right: a 3-d view (Delaunay-triangulated surface through the training points).

The response surface is smooth and has a clear structure:

- Along x_2 it is roughly **symmetric and U-shaped**. y is smallest near $x_2 = 0$ and grows towards $x_2 = \pm 3$.
- Along x_1 it **oscillates once**. There is a valley around $x_1 \approx -1.5$, where the minimum $y \approx -1$ is reached near $(x_1, x_2) \approx (-\pi/2, 0)$, and a ridge around $x_1 \approx +1.5$, where the largest values $y \approx 5.5$ occur in the corners $(\approx \pi/2, \pm 3)$.

This looks like $y \approx \sin(x_1) + c\,x_2^2$ plus noise. An exploratory least-squares fit of $y = \beta_0 + \beta_1 \sin x_1 + \beta_2 x_2^2$ on the training set gives $(\beta_0, \beta_1, \beta_2) = (-0.01,\ 1.01,\ 0.50)$ and a *validation* MSE of 0.0353. So the data are close to $y = \sin(x_1) + \frac{1}{2}x_2^2 + \varepsilon$ with $\mathrm{Var}(\varepsilon) \approx 0.034$. I use 0.035 as a rough *noise floor*

for the validation MSE when judging the flexible models below. They are not given this functional form.

(b) Random Kitchen Sinks with ridge regression

Model. For $j = 1, \dots, h$ draw $w_j \sim \mathcal{N}(0, I_2/\sigma^2)$ and $b_j \sim \mathcal{U}[0, 2\pi]$, and map

$$z(x) = \big(\sin(w_1^\top x + b_1), \cos(w_1^\top x + b_1), \dots, \sin(w_h^\top x + b_h), \cos(w_h^\top x + b_h)\big)^\top \in \mathbb{R}^D, \qquad D = 2h.$$

On top of these random features I fit ridge regression with an unpenalized intercept,

$$\min_{\beta_0, \beta} \ \frac{1}{n} \sum_{i=1}^{n} \big(y_i - \beta_0 - \beta^\top z(x_i)\big)^2 + \lambda \|\beta\|_2^2.$$

Centring Z and y removes β_0. Setting the gradient to zero gives the closed form

$$\hat{\beta} = \big(Z_c^\top Z_c + n\lambda I_D\big)^{-1} Z_c^\top y_c, \qquad \hat{\beta}_0 = \bar{y} - \bar{z}^\top \hat{\beta},$$

where $Z_c = Z - \mathbf{1}\bar{z}^\top$ and $y_c = y - \bar{y}\mathbf{1}$. The problem is strictly convex, so this is the unique global optimum. No iterative training is needed.

Why σ matters (kernel view). Using $\sin a \sin b + \cos a \cos b = \cos(a - b)$,

$$\frac{1}{h}\, z(x)^\top z(x') = \frac{1}{h} \sum_{j=1}^{h} \cos\big(w_j^\top (x - x')\big) \xrightarrow[h \to \infty]{\text{a.s.}} \mathbb{E}_w\big[\cos(w^\top \delta)\big] = \exp\Big(-\frac{\|x - x'\|^2}{2\sigma^2}\Big), \qquad \delta = x - x',$$

by the law of large numbers ($w_1, \dots, w_h$ are i.i.d. and $|\cos| \le 1$). Here $\mathbb{E}[\cos(w^\top \delta)]$ is the characteristic function of $\mathcal{N}(0, I/\sigma^2)$ evaluated at δ. (The phases b_j cancel when sin/cos pairs are used.) RKS with ridge is therefore a Monte-Carlo approximation of **Gaussian-kernel ridge regression with bandwidth σ**. A small σ means high-frequency features and a very wiggly function class, and a large σ means low-frequency features and a very smooth function class. So σ directly controls the bias–variance trade-off. I therefore sweep σ.

Sweep. I fix $D = 1000$ ($h = 500$) and $\lambda = 10^{-3}$, and vary σ over 16 log-spaced values in $[0.05, 10]$. Each value is repeated over 5 random draws of (w_j, b_j), and I report mean $\pm$ sd.

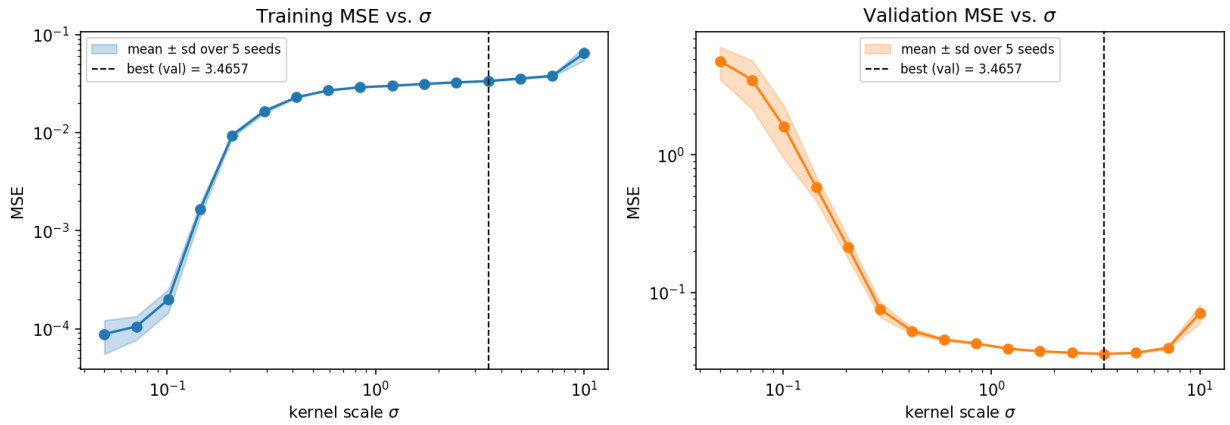


Figure 2: RKS: training (left) and validation (right) MSE vs. kernel scale σ ($D = 1000$, $\lambda = 10^{-3}$; mean $\pm$ sd over 5 seeds; log axes).

σ	0.05	0.10	0.21	0.42	0.84	1.71	**3.47**	10.0
train MSE	0.0001	0.0002	0.0094	0.0229	0.0290	0.0314	0.0336	0.0646
val MSE	4.806	1.598	0.213	0.0524	0.0426	0.0374	**0.0358**	0.0706

Table 1: Selected points of the RKS sweep (means over 5 seeds; all 16 values are in `problem3_rks_sweep.csv`).

The sweep shows the classic U-shape.

- **Small σ (overfitting).** The kernel bandwidth σ sets the length scale over which the fit can change. For $\sigma \leq 0.1$ it is below the typical spacing of the training points ($\sqrt{36/600} \approx 0.25$), so each point gets its own narrow bump. With 1000 features, only 600 points and a tiny λ, the model interpolates: training MSE is about 10^{-4}, while validation MSE is 1.6–4.8, worse than predicting a constant.
- **Large σ (underfitting).** At $\sigma = 10$ every feature varies slowly across $[-3, 3]^2$. A combination of such features can still produce curvature, but only with large, partially cancelling coefficients, and the ridge penalty (with λ fixed) suppresses exactly those. Both errors rise to about 0.07 (the fit still has $R^2 \approx 0.97$ on validation, so this is a moderate underfit, not a failure).
- **Best:** $\sigma^\star = 3.47$, with validation MSE 0.0358 ± 0.0001 (the $\pm$ is the sd over the 5 random feature draws only; see (d) for the much larger uncertainty from the validation sample). This is essentially at the noise floor (≈ 0.035). The training MSE at $\sigma^\star$ (0.034) is also about the noise variance, which is what a well-regularized model should show.

(c) One-hidden-layer MLP

Model and training. $f(x) = v^\top \mathrm{ReLU}(Wx + c) + b$ with $W \in \mathbb{R}^{H \times 2}$, in PyTorch. I train with MSE loss and Adam (learning rate 3×10^{-3} with cosine decay to 0), batch size 64, and 600 epochs,

without early stopping or weight decay. Unlike RKS, the hidden layer is *learned*, and training is a non-convex optimization.

Sweep. I vary the hidden width $H \in \{1, 2, 4, \ldots, 1024\}$ (11 values) and repeat each over 5 random initializations.

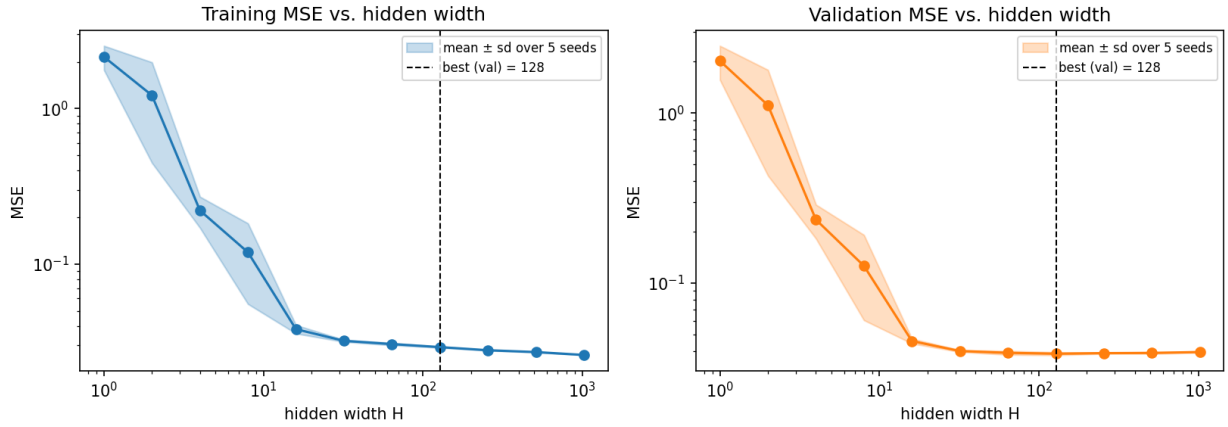


Figure 3: MLP: training (left) and validation (right) MSE vs. hidden width H (mean ± sd over 5 seeds; log axes).

H	1	2	4	8	16	32	**128**	512	1024
train MSE	2.153	1.217	0.221	0.119	0.0381	0.0321	0.0292	0.0272	0.0260
val MSE	2.023	1.110	0.236	0.126	0.0457	0.0399	**0.0385**	0.0389	0.0394

Table 2: Selected points of the MLP width sweep (means over 5 seeds; full results in `problem3_mlp_sweep.csv`).

- **Narrow networks underfit.** With $H \leq 8$ ReLU units, the function is piecewise linear with very few pieces, which cannot represent the curved surface. Results also vary strongly with the initialization (large error bands): with very few units, some seeds end with "dead" or badly placed units.
- From $H \approx 32$ onwards the validation MSE is flat at about 0.039. The best value is $H^\star = 128$ (validation MSE 0.0385 ± 0.0008).
- **Very wide networks overfit only mildly.** Training MSE keeps decreasing (0.026 at $H = 1024$, below the noise variance ≈ 0.034), so the network starts fitting noise. Validation MSE rises only slightly ($0.0385 \to 0.0394$). With a fixed training budget, the implicit regularization of (stochastic) gradient descent keeps the wide networks from interpolating the data.

(d) Comparison and visualization

Refit and validation loss. Both models were refit on the training set with the selected hyperparameters (seed 0):

Model	hyper-parameters	# trained params	train MSE	**val MSE**
Constant (train mean)	—	1	2.463	2.397
RKS + ridge	$\sigma = 3.47$, $D = 1000$, $\lambda = 10^{-3}$	1,001	0.0333	**0.0357**
1-hidden-layer MLP	$H = 128$, ReLU, Adam	513	0.0293	0.0388
Reference: least squares on $\{1, \sin x_1, x_2^2\}$ (true functional form)				0.0353

The test-set predictions are in `rks_test_predictions.csv` and `mlp_test_predictions.csv`, one value per row in the order of `test.csv`. The two models agree closely on the test inputs: the mean squared difference between their predictions is 0.0026.

Which is better and why. Both models capture the function well: about 98.5% of the validation variance is explained. RKS has the lower validation MSE (0.0357 vs. 0.0388), essentially at the noise floor (0.0353). With only 200 validation points, each validation MSE has a standard error of about 0.003 (0.0035 for RKS, 0.0037 for the MLP, computed from the per-point squared errors). The ± values in the sweep tables reflect only the randomness of the features or the initialization, not the sampling of the validation set. Compared naively, the gap of 0.0031 would be only about 0.6 (unpaired) standard errors (0.0051).

Both models are, however, scored on the *same* validation points, and their squared errors are highly correlated across points (correlation 0.94): the points that are hard for one model are hard for the other, because most of the error is the irreducible noise ε_i. The right comparison is therefore paired. With $d_i = (y_i - \hat{f}_{\mathrm{MLP}}(x_i))^2 - (y_i - \hat{f}_{\mathrm{RKS}}(x_i))^2$,

$$\bar{d} = 0.0031, \qquad \widehat{\mathrm{SE}}(\bar{d}) = \frac{s_d}{\sqrt{200}} = 0.0012, \qquad t \approx 2.5.$$

On the same validation points, the paired difference is 0.0031 ± 0.0012, which *is* clearly larger than its standard error (about 2.5 SE). The gap also has the same sign for all five random seeds of both models ($0.0031, 0.0036, 0.0026, 0.0015, 0.0027$; mean 0.0027). So there is reasonably good evidence that RKS fits this function slightly better, but the difference is small in absolute terms, about 8% of the MSE, and both models are close to the noise floor.

Two caveats apply. First, $\sigma^\star$ and $H^\star$ were both selected on this validation set, so both validation MSEs are slightly optimistic. Both validation curves are very flat near their minima (Figures 2 and 3), so the selection bias should be small and similar for the two models. Second, the noise-floor reference (0.0353) is estimated on the same 200 points and has a similar standard error, so "at the noise floor" should be read as "not distinguishable from it".

Plausible reasons why RKS comes out slightly ahead are:

1. **Right inductive bias.** RKS with $\sigma = 3.47$ approximates a Gaussian-kernel ridge regression whose function class consists of very smooth (infinitely differentiable) functions. The true function $\sin x_1 + \frac{1}{2}x_2^2$ is exactly of this kind: smooth, low-frequency, and without kinks. A ReLU network is *piecewise linear*. It has to approximate curvature with many flat pieces joined at kinks, and with only 600 noisy points some kinks end up in the wrong place.

2. **Convex, closed-form fit.** The random features are fixed, so ridge regression gives the exact global optimum, and its regularization (λ, σ) is explicit and easy to tune. The MLP solves a non-convex problem with SGD noise. Its effective regularization comes implicitly from the optimizer and training length, and its training MSE (0.029) is already below the noise variance, i.e. it fits some noise.

3. **Low dimension.** In $d = 2$, 500 random directions cover all orientations densely, and because the target is smooth, slowly varying features suffice to build it. The main advantage of the MLP, *learning* which features to use, matters most in high dimensions or for functions with localized or non-smooth structure, and neither is the case here.

The MLP's strengths show up in other ways: it reaches nearly the same accuracy with about half as many trained parameters (513 vs. 1001), and its accuracy is insensitive to the width once $H \geq 32$, while RKS fails badly for σ that is too small.

Fitted surfaces. Figures 4 and 4 were made with the provided `plot_surface_2d`. Figure 5 shows the training data and both fits on a common colour scale, together with their difference.

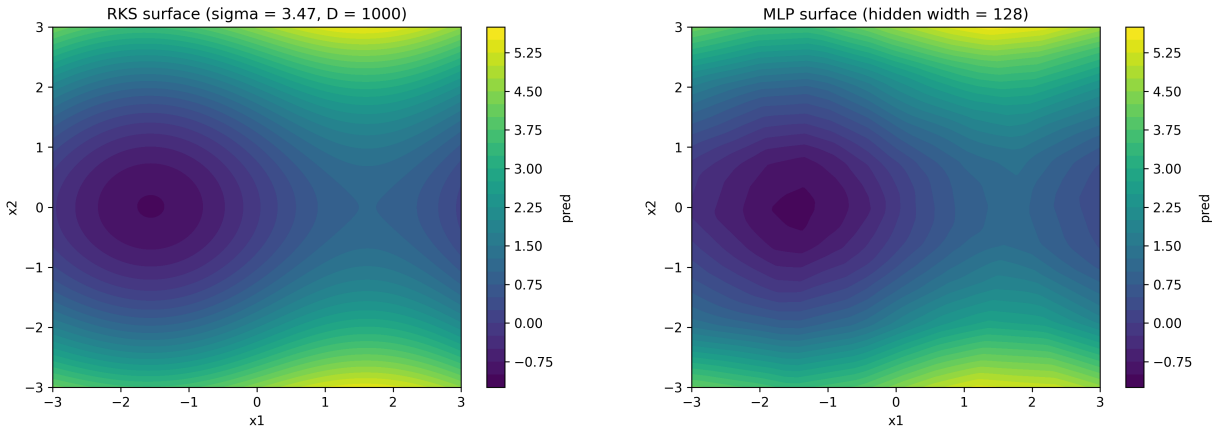


Figure 4: Fitted surfaces from `plot_surface_2d`. Left: RKS ($\sigma = 3.47$). Right: MLP ($H = 128$).

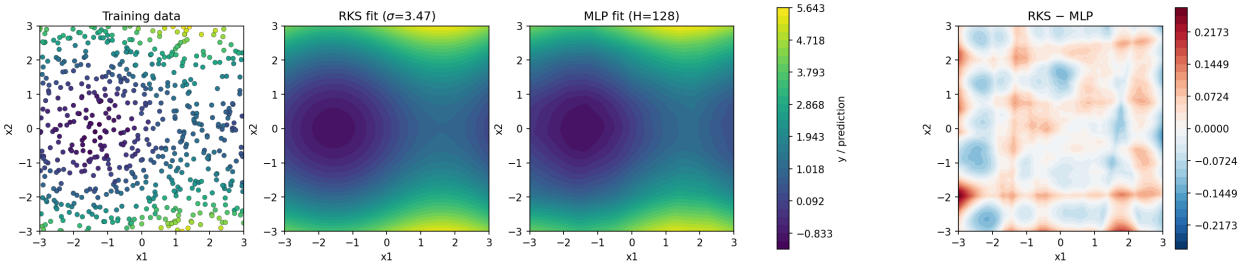


Figure 5: From left to right: training data; RKS fit; MLP fit (shared colour scale); pointwise difference RKS − MLP.

How each model captures the structure. Compared with the data in (a), both surfaces reproduce the global structure: the valley at $(x_1, x_2) \approx (-\pi/2, 0)$, the ridge along $x_1 \approx \pi/2$, the symmetric parabolic growth in $|x_2|$, and the maxima in the corners $(\pi/2, \pm 3)$. The surfaces are much smoother than the triangulated data in Figure 1, i.e. both models average out the observation noise instead of chasing individual points. The two models differ only in the details:

- The **RKS** surface is a weighted sum of 1000 smooth sinusoids. At $\sigma^\star = 3.47$ their typical wavelength $2\pi/\|w_j\|$ is about 17, much longer than the width of the domain. Since $\sigma\|w_j\|$ is

Rayleigh-distributed in $d = 2$, only a fraction $P(\|w_j\| \geq 1) = e^{-\sigma^2/2} \approx 0.25\%$ of the features (about one of the 500 frequencies) oscillates at least as fast as $\sin x_1$. The oscillation in x_1 is therefore not copied from any single feature but built by combining many slowly varying ones, which is why the level sets are smooth, rounded curves, as expected for a Gaussian-kernel smoother.

- The **MLP** surface is a sum of 128 ReLU "hinges". Each one is linear on either side of a line $w_k^\top x + c_k = 0$. Its contours are made of nearly straight segments with slight corners. This is clearest in the valley, which comes out more polygonal (diamond-shaped) than round, and in the difference plot (right panel of Figure 5), which shows straight-line artefacts along the learned hinge directions.

- The two fits differ by at most about 0.3, and the largest differences are at the **edges and corners** of the square (e.g. near $(-3, -2)$), where data are sparse and each model extrapolates according to its own inductive bias. A ReLU network continues linearly, while the RKS fit continues smoothly. In the interior, where the data are dense, the differences are mostly below 0.1, well under the noise standard deviation (≈ 0.18).